Jeu de Match 3

Objectif du projet (6h TD, 4h TDAO, 14h TP)

L'objectif du projet est de mettre en œuvre les notions et outils de la programmation orientée objet (classes, algorithmes, tableaux, listes, fichiers, interfaces graphiques) sur un problème concret et original donnant lieu à la réalisation d'un petit logiciel.

Les parties du projet abordées en TD constituent le socle *minimum* du projet. Il vous appartient ensuite de poursuivre votre projet par du travail personnel dans la direction de votre choix. Quelques pistes d'extensions sont données à titre d'exemple à la fin du sujet.

I Cadrage du projet

1 Contraintes et attendus

Les contraintes suivantes sont imposées :

1. le projet doit être réalisé en binôme (ou seul avec l'accord de votre enseignant de TP) ;
2. le projet doit être réalisé en Java avec Netbeans ;
3. le projet doit utiliser les notions et outils abordés dans le module ;
4. le projet doit comporter une interface permettant de tester rapidement les principales fonctionnalités du logiciel réalisé.

Le projet doit être rendu sous forme d'une archive compressée **zip** contenant :

- tous les fichiers du projet `NetBeans` (répertoire complet du projet `NetBeans`) ;
- un rapport en pdf (voir plus loin).

Le fichier **zip** doit s'intituler `NOM1-NOM2.zip` et doit être déposé sur **moodle**. La date butoir pour rendre le projet est indiquée sur **moodle**.

2 Programme de travail

TD (2h) Analyse des besoins et conception architecturale.

TP (6h) Première partie du projet.

Travail personnel

TD (4h) Approfondissements.

TP (4h) Deuxième partie de projet.

Travail personnel

TDAO (4h) Découverte des interfaces graphiques.

TP (4h) Réalisation d'une interface graphique permettant l'utilisation des algorithmes réalisés.

Travail personnel

3 Évaluation

L'évaluation du projet tient compte des éléments suivants :

1. résultats obtenus : étendue des fonctionnalités programmées, justesse des résultats, absence de bugs, etc. ;
2. utilisation adaptée des notions abordées dans le module : classes, structures de contrôle, tableaux, listes, arbres, récursivité, fichiers, interfaces graphiques, etc. ;
3. qualité de la programmation : lisibilité du programme (utilisation de noms pertinents pour les classes, les attributs, les méthodes et les variables), architecture du programme (découpage en différentes classes, répartitions des traitements dans les différentes classes, utilisation et ré-utilisation des méthodes), présence de tests ;
4. qualité du rapport écrit ;
5. qualité de la présentation orale ;
6. comportement en séance de TP (préparation, implication, sérieux, etc.).

4 Contenu du rapport

Un rapport d'une dizaine de pages doit accompagner le projet `NetBeans` et doit permettre à un informaticien ne connaissant pas votre sujet de comprendre le travail effectué. Par exemple, un autre groupe d'étudiant devrait être capable de poursuivre le projet.

La rapport doit présenter la démarche suivie pendant la réalisation de votre projet. Il pourra notamment lister les différentes fonctionnalités réalisées, décrire les classes Java sous la forme d'un diagramme de classes UML, présenter un algorithme participant à la réalisation d'une fonctionnalité importante du logiciel, donner un exemple d'utilisation du logiciel avec des captures d'écran.

Le rapport devra également inclure une conclusion présentant un bilan et une analyse critique du travail effectué, les limites du logiciel, ainsi que les perspectives d'amélioration ou d'extension. Dans le bilan, il est possible d'inclure un aspect de « répartition des tâches » si le projet a été réalisé en binôme.

II Sujet du projet : Jeu de Match 3

Le jeu vidéo Match 3 tourne autour d'un principe simple : le joueur doit manipuler des tuiles sur une grille pour créer une combinaison de tuiles (par exemple un alignement d'au moins 3 éléments identiques). Lorsqu'une combinaison est créée, elle disparait et les tuiles situées au dessus tombent en déclenchant éventuellement d'autres combinaisons. Malgré sa simplicité ce type de jeu est devenu une forme de divertissement numérique les plus populaires et les plus accessibles.

Les origines des jeux de Match 3 remontent aux premiers jours des jeux numériques, mais ce n'est que dans les années 1990 que le format a commencé à prendre la forme que nous connaissons aujourd'hui. Le jeu "Bejeweled", sorti en 2000 par PopCap Games, est un des tout premier jeu de match connu.

Figure 1.1 – Jeu "Bejeweled 3"

La complexité et la variété des jeux de Match 3 a grandement augmentée aujourd'hui avec notamment le titre "Candy Crush Saga" de King qui introduit de nouveaux éléments de jeu tels que des niveaux, des obstacles et des bonus.
Le projet a pour objectif de réaliser un programme capable de proposer un jeu de Match 3.

1 Première partie : jouer en mode console

Le but de la première partie est de programmer un clone du jeu dans une version où la seule combinaison qui élimine des tuiles est celle de 3 tuiles identiques alignées. L'objectif est de se familiariser avec les règles et de mettre en place une modélisation objet adaptée. Le jeu a au moins deux paramètres, la taille du plateau de jeu et le nombre de tuiles différentes présentes sur le plateau. On veillera à ce le jeu dans son état initial ne présente pas de combinaison (qui devrait être éliminée).

Dans la première version du jeu, seul un affichage texte est requis. On attend les fonctionnalités suivantes :

1. Recherche de la position d'un match (combinaison avec 3 tuiles identiques alignées horizontalement ou verticalement).
2. Suppression des tuiles qui "matchent" et "chute" des tuiles situées au dessus avec ajout aléatoire de nouvelles tuiles par le haut.
3. Action par échange de tuiles voisines (saisie des coordonnées au clavier).

Des classes `Clavier` et `Outils` sont fournies pour faciliter les saisies au clavier et les affichages console en couleur.
Un exemple d'affichage en mode console est donnée dans la figure ci-dessous

Cette partie du projet dans sa version console doit être terminée à l'issue des 6 premières heures de TP consacrées au projet. Si ce n'est pas le cas, vous devez terminer le programme chez vous.

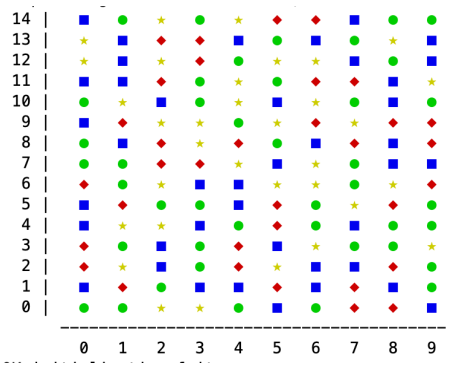

FIGURE 1.2 – Jeu en mode console

2 Deuxième partie : l'ordinateur propose des échanges

Dans cette seconde partie, nous ajoutons un peu d'intelligence au jeu. Le but est que l'ordinateur identifie les actions possibles.

1. On ajoute une fonctionnalité qui permet de vérifier si il existe des échanges encore possibles.
2. On ajoute une fonctionnalité qui liste tous les échanges possibles.
3. On ajoute une fonctionnalité qui permet à l'ordinateur de choisir l'échange le plus intéressant (celui qui permet de marquer le plus de points possibles, en prenant par exemple 300 points par match). et de jouer automatiquement N coups.

3 Troisième partie : jouer en mode graphique

Enfin, on crée un jeu jouable avec une interface graphique.

1. Affichage graphique de l'état du jeu 1.3.

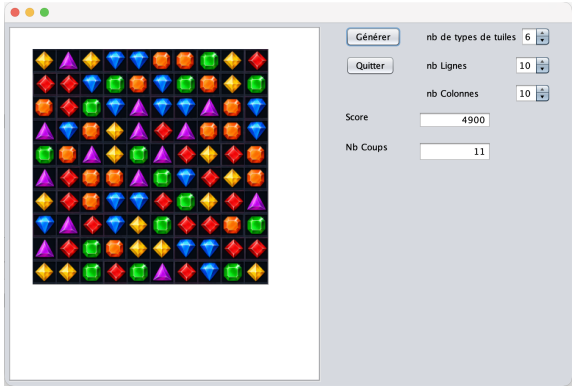

FIGURE 1.3 – Jeu en mode graphique

2. Interactions avec la souris (pour réaliser les échanges à partir de 2 clics souris). Attention il faudra vérifier les positions des tuiles échangées et la création d'un match
3. On compte les points marqués et on limite le nombre de coups maximum d'une partie.
4. Ajout de nouvelle combinaisons de match telle que sur la figure 1.4. Les points marqués pour des matchs à 4 tuiles ou à 5 tuiles augmentent, respectivement à 500 points et 1000 points.

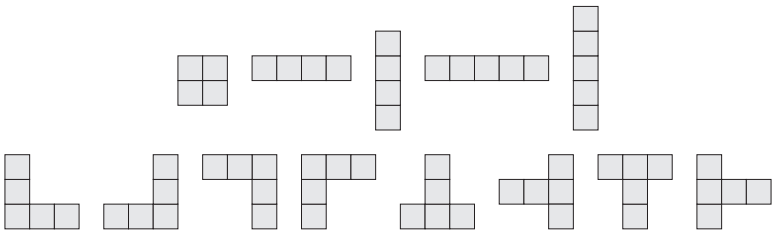

Figure 1.4 – combinaisons de match à 4 et 5 tuiles

5 On pourra ensuite ajouter éventuellement des bonus :

- En associant quatre tuiles de la même couleur dans une rangée, un bonus Ligne permet de retirer toutes les tuiles dans une rangée ou une colonne
- En associant cinq tuiles de la même couleur en T ou en L, un bonus Bombe qui permet d'éliminer toutes les cellules adjacentes
- En associant cinq tuiles de la même couleur sur une rangée, un bonus Super qui permet de retirer toutes les tuiles d'une couleur du plateau